



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECP3623
Database Programming
Semester 1 2025/2026
Assignment 2 (10%)

Lecturer: Shamini A/P Raja Kumaran

Name	:	YASMIN BATRISYIA BINTI ZAHIRUDDIN
Matric Id	:	A23CS0201
Section	:	02

Name	:	NURUL ADRIANA BINTI KAMAL JEFRI
Matric Id	:	A23CS0258
Section	:	02

Question 1

i)

X.

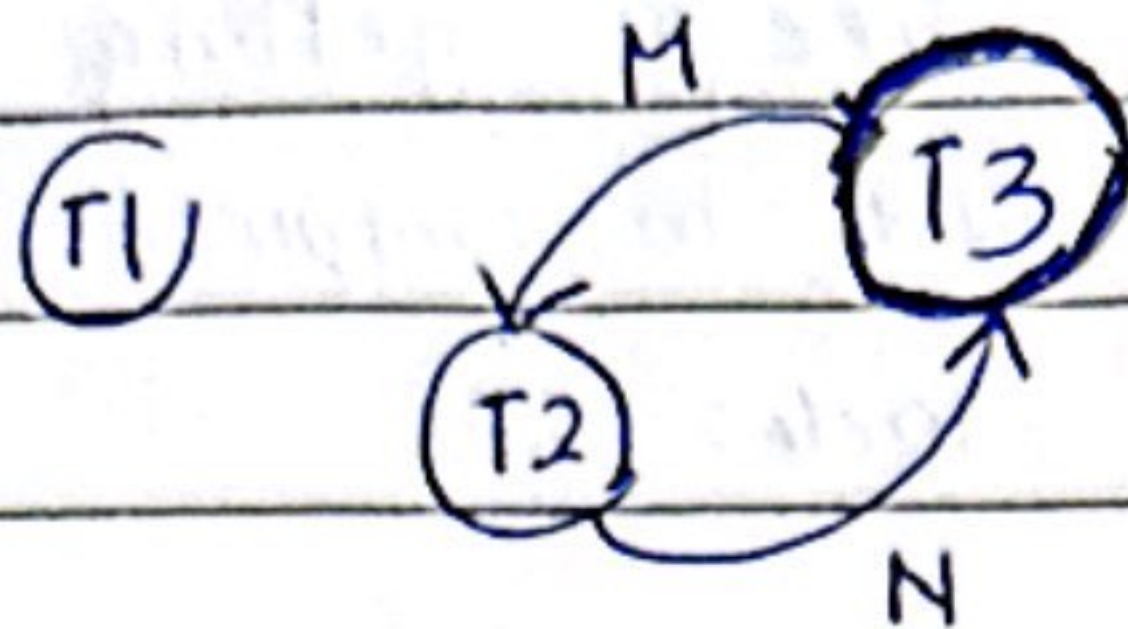
Y.

M.

N.

1) R(x)

1) R(y)

1) R(M) $\rightarrow$ R(M) (X)1) R(N) $\rightarrow$ R(N) (X)2) R(M) $\rightarrow$ W(M) (X)2) R(N) $\rightarrow$ W(N) (✓)3) R(M) $\rightarrow$ W(M) (✓)3) W(N) $\rightarrow$ R(N) (X)

ii) cycle = yes

conflict = no

Therefore, precedence graph has a cycle, so is not serializable.

iii)

time

T1

T2

T3

 t_1

begin transaction

 t_2

read-lock item(x)

 t_3

read item(x)

begin transaction

 t_4

read-lock item(y)

read-lock item(M)

begin transaction

 t_5

read item(y)

read item(M)

read-lock item(N)

 t_6

commit

read-lock item(N)

read item(N)

 t_7

read item(N)

read-lock item(M)

 t_8 $M = (N + M) * 10$

WAIT

 t_9

write-lock item(M)

WAIT

 t_{10}

write-item(M)

WAIT

 t_{11}

commit

read item(M)

 t_{12} $N = N + 150$ t_{13}

write-lock item(N)

 t_{14}

write item(N)

 t_{15}

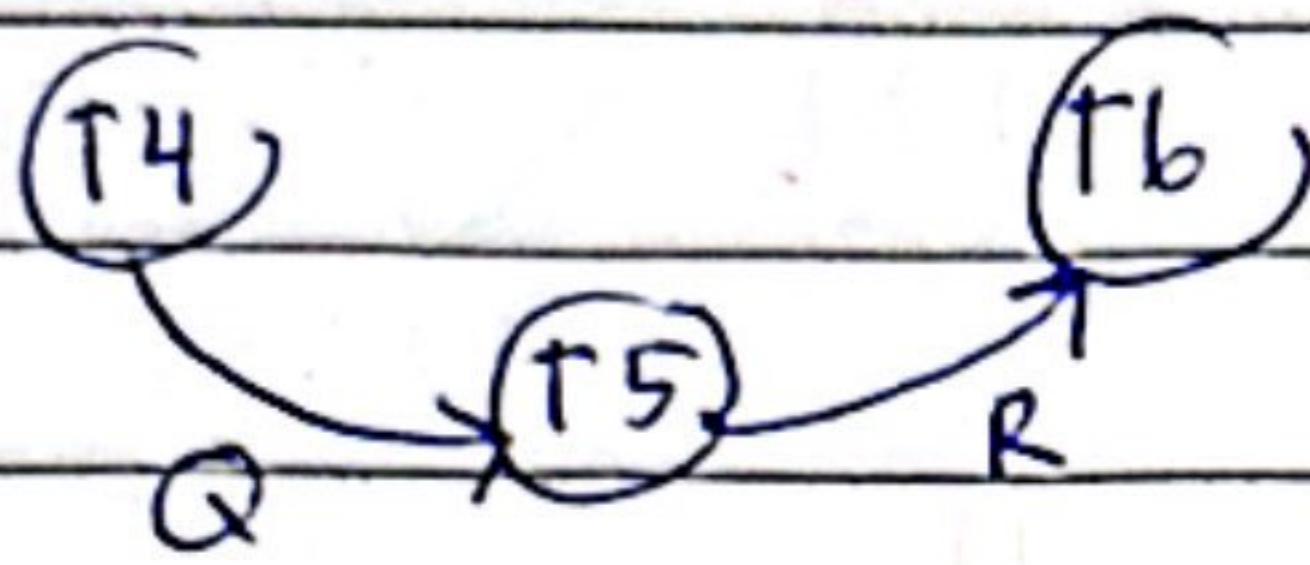
commit

iv) Problem that acquire is blocking where T3 must wait until T2 release lock M and N. Next is reduced concurrency because transaction cannot interleave freely.

Potential problem is deadlock if T2 lock M and T3 lock N first, so both may wait forever. Next, is starvation, a transaction may wait too long if others keep getting locks. Lastly, livelock where no transaction are able to complete their task, and they cannot acquire any new locks.

Question 2

i)

P.1) $R(P) \xrightarrow{T4} W(P)$ (X)Q.1) $R(Q) \xrightarrow{T4} R(Q)$ (X)2) $R(Q) \xrightarrow{T5} W(Q)$ (X)3) $R(Q) \xrightarrow{T4} W(Q)$ (✓)R.1) $R(R) \xrightarrow{T5} R(R)$ (X)2) $R(R) \xrightarrow{T5} W(R)$ (✓)3) $R(R) \xrightarrow{T6} W(R)$ (X)

ii)

cycle = no

conflict = yes

∴ precedence graph has no cycle, so it has
~~conflict~~ serializable.

ii)

time

T4

T5

T6

 t_1

begin transaction

 t_2

read-lock item(P)

 t_3

read item(P)

begin transaction

 t_4

read-lock item(Q)

read-lock item(Q)

 t_5

read item(Q)

read item(Q)

begin transaction

 t_6 $P = (P \times 5) + Q$

read-lock item(R)

read-lock item(R)

 t_7

write-lock item(P)

read item(R)

read-item(R)

 t_8

write item(P)

 $Q = Q + R$ $R = R - 100$ t_9

commit

write-lock item(Q)

write-lock item(R)

 t_{10}

write item(Q)

write item(R)

 t_{11}

commit

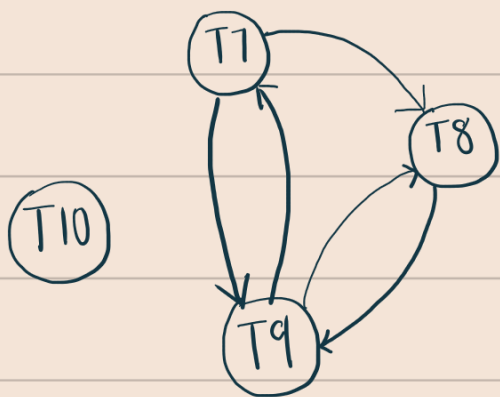
commit.

Question 3

a) i] **non-serial schedule**. The operations of transactions T7, T8, T9 and T10 are interleaved.

The transactions do not execute one after another in a strictly sequential order and overlap in time.

ii]



iii] The schedule is **not conflict-serializable**.

Conflict serializability is determined by checking whether the precedence graph contains a cycle. In this case, there is a cycle between T7 and T9 — meaning

the precedence graph is cyclic.

iv] The schedule faces a **lost update**

problem due to multiple transactions

(T7 and T9) update the same data

item X without proper synchronization. Updates made by one transaction can be overwritten

by another, causing some updates to be lost. Also, the lack of concurrency control may also

lead to inconsistent data values.

b) i]	Time	T ₁₁	T ₁₂	T ₁₃	T ₁₄
	t ₁	begin transaction			
	t ₂	lock - X (A)	begin transaction		
	t ₃	read (A)	lock - X (B)	begin transaction	
	t ₄		read(B)	lock - X (A)	begin transaction
	t ₅	A = A + 200	B = B - 50	WAIT	lock - X (C)
	t ₆	write (A)		WAIT	read (C)
	t ₇	unlock (A)	write (B)	WAIT	C = C + 10
	t ₈		unlock (B)	read (A)	write (C)
	t ₉				unlock (C)
	t ₁₀	commit		A = A - 30	commit
	t ₁₁		lock - X (A)	write (A)	
	t ₁₂		WAIT	unlock (A)	
	t ₁₃		read (A)		
	t ₁₄		A = A * 2		
	t ₁₅		write (A)		
	t ₁₆		unlock (A)		
	t ₁₇		commit	commit	
	t ₁₈				

ii] The schedule is non-serializable due to the overlap transactions. Therefore,

concurrency problem may still occur even when all transactions commit successfully

iii]	Time	T ₁₁	T ₁₂	T ₁₃	T ₁₄
	t ₁	begin transaction			
	t ₂	lock-X(A)	begin transaction		
	t ₃	read(A)	lock-X(B)	begin transaction	
	t ₄		read(B)	lock-X(A)	begin transaction
	t ₅	A = A + 200	B = B - 50	WAIT	lock-X(C)
	t ₆	write(A)		WAIT	read(C)
	t ₇	commit/unlock(A)	write(B)	WAIT	C = C + 10
	t ₈			read(A)	write(C)
	t ₉		lock-X(A)		commit/unlock(C)
	t ₁₀		WAIT	A = A - 30	
	t ₁₁		WAIT	write(A)	
	t ₁₂		WAIT	commit/unlock(A)	
	t ₁₃		read(A)		
	t ₁₄		A = A * 2		
	t ₁₅		write(A)		
	t ₁₆		commit/unlock(A,B)		

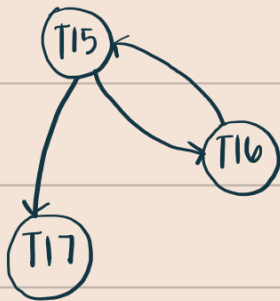
iv] **cascading rollback**, other dependent transactions such as T₁₂ and T₁₃ will roll back if T₁₁ fails after unlocking A. **Starvation** may happen too because it needs to wait for the previous transaction to finish and unlock all resources.

Question 4

i] **Non-serial schedule**, because the operations of the three transactions (T_{15}, T_{16}, T_{17}) are interleaved.

For example, T_{15} begins and reads J but before it completes, T_{16} begins and operates on K and T_{17} operates on J . In a serial schedule, one transaction must commit before the next one begins.

ii]



iii] No, it is **not conflict-serializable**. There is a cycle involving

T_{15} and T_{16} due to the conflicts on data item K as shown in the precedence graph. Therefore, the schedule cannot be serialized since this precedence graph is not acyclic.

iv] The schedule faces the **lost update** problem and **uncommitted dependency** (dirty read).

Lost update happens on K : T_{16} updates K at t_5 but T_{15} also updates K at t_7 based on a value read at t_5 . Since T_{15} overwrites T_{16} 's write without considering it, T_{16} 's update is lost. Meanwhile, uncommitted dependency happens on J : T_{17} reads J at t_4 immediately after T_{15} writes J at t_4 . Since T_{15} has not committed yet, T_{17} is reading dirty (uncommitted) data.

v]	Time	T_{15}	T_{16}	T_{17}
	t_1	begin transaction		
	t_2	lock-X(J)	begin transaction	

t_3	read(J)	lock-X(K)	begin transaction
t_4	$J = J + 10$	read(K)	lock-X(J)
t_5	write(J)	$K = K - 5$	WAIT
t_6	lock-X(K)	write(K)	WAIT
t_7	WAIT	commit/unlock(K)	WAIT
t_8	lock-X(K)		WAIT
t_9	read(K)		WAIT
t_{10}	$K = K + 20$		WAIT
t_{11}	write(K)		WAIT
t_{12}	commit/unlock(K, J)		WAIT
t_{13}			lock-X(J)
t_{14}			read(J)
t_{16}			$J = J - 15$
t_{17}			write(J)
t_{18}			commit/unlock(J)